# Return values and local state

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

---

WORKBOOK · 1 OF 15

## Recall and choose your starting point

> Record your starting explanation for Return values and local state before opening notes. Show any later correction separately.

Try the recall before opening notes. Then use a different colour or a labelled second line to correct your explanation.

### Functions and arguments

Without opening the old lesson, explain one key idea from Functions and arguments. Give an example and a mistake that the idea helps you avoid.

> **My remembered idea, example and correction**

### Loops and trace tables

Without opening the old lesson, explain one key idea from Loops and trace tables. Give an example and a mistake that the idea helps you avoid.

**My remembered idea, example and correction**

### Expressions and operator order

Without opening the old lesson, explain one key idea from Expressions and operator order. Give an example and a mistake that the idea helps you avoid.

**My remembered idea, example and correction**

---

WORKBOOK · 2 OF 15

## Practise with fading help

> Complete the supported Return values and local state trace and record the first step you can explain without help.

Use W2: Trace separate calls for **Return value**s and local state. Record the answer once; the lesson and workbook refer to the same attempt.

| Given inputs | Procedure I chose | Middle steps | Result and check |
|---|---|---|---|
| | | | |

**The step I completed without help, and the hint I needed**

WORKBOOK · 3 OF 15

## Individual reasoning check

> Check the proposed Return values and local state explanation against the supplied facts before discussing it.

Use the worked case for **Return value**s and local state. Proposed explanation: “Printing a value returns that value”. Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

**My explanation before partner discussion**

WORKBOOK · 4 OF 15

## Independent transfer case

> Write one first attempt at the reserved independent case before feedback. Repeated copies of that case refer to the same recorded attempt.

### Independent case: Use returns and keep local state local

> Trace both calls and the final expression. If return total is removed from the function, does this final print complete? Would a separate print(total) outside the function have access to its **local variable**?

```
def twice(n):
    total = 2 * n
    return total
first = twice(3)
second = twice(5)
print(first, second, first + second)
```

> **My independent explanation, trace and limits**

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

---

WORKBOOK · 5 OF 15

## W1 Separate returning from displaying

> Separate returning from displaying. Apply the supplied Return values and local state facts and keep your reasoning.

### Where to find the supplied example

Reference for this exercise: open this week's student lesson, "Return values and local state", at "See how it works" and "Follow a worked example". Use the original worked values or code named in the question; the later pictorial case is a separate example. Keep this reference beside your workbook while practising.

Using the correct worked definition, execute result = remaining(15, 6), then doubled = result * 2. State both stored values and what the script displays before a print statement is added.

**My reasoning and evidence**

---

WORKBOOK · 6 OF 15

## W2 Trace separate calls

Trace separate calls. Apply the supplied Return values and local state facts and keep your reasoning.

### Where to find the supplied example

Reference for this exercise: open this week's student lesson, "Return values and local state", at "See how it works" and "Follow a worked example". Use the original worked values or code named in the question; the later pictorial case is a separate example. Keep this reference beside your workbook while practising.

Using the correct definition, store a = remaining(8, 3), b = remaining(8, 8), c = remaining(8, 3). State a, b and c. In a fresh script, explain why print(spaces) outside the function fails.

**My reasoning and evidence**

---

WORKBOOK · 7 OF 15

## W3 Repair the returned value

> Repair the returned value. Apply the supplied Return values and local state facts and keep your reasoning.

### Where to find the supplied example

Reference for this exercise: open this week's student lesson, "Return values and local state", at "Find and repair a mistake". Use the original worked values or code named in the question; the later pictorial case is a separate example. Keep this reference beside your workbook while practising.

Use the faulty printing version with answer = remaining(11, 2). State the displayed number and the value stored in answer. Explain why print(answer + 1) fails, then repair the function and predict the repaired **output**.

**My reasoning and evidence**

WORKBOOK · 8 OF 15

## W4 Build a reusable calculation independently

> Attempt before feedback. If this repeats the selected independent case, reuse that first attempt; otherwise keep a separate record. Build a reusable calculation independently. Apply the supplied Return values and local state facts and keep your reasoning.

Each tray holds 5 counters. Define stock(trays, loose) to return the total using a **local variable** total. Assume nonnegative integer arguments. Store results for (2, 3), (0, 3) and (2, 0).

Print the sum of the first two stored results. Predict that **output** and all three stored values. Call (2, 3) again to check fresh state.

**My reasoning and evidence**

WORKBOOK · 9 OF 15

## W5 Locate a statement that never runs

> Locate a statement that never runs. Apply the supplied Return values and local state facts and keep your reasoning.

A function body has spaces = capacity - used, then return spaces, then print("done"), all at the same indentation. What does answer = remaining(4, 4) store and display? Explain whether the final print can run.

**My reasoning and evidence**

WORKBOOK · 10 OF 15

## Project checkpoint and reflection

> Connect Return values and local state to Python readiness portfolio, identify a limit and choose the next useful test.

Add a labelled **return value**s and local state **evidence** sheet to Python readiness portfolio. Show how you can use a returned value in a later calculation.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

**My project change, supporting evidence and remaining limit**

WORKBOOK · 11 OF 15

## Feedback, return check and learning record

> Record the first repaired step and a different case to revisit after a delay.

| Evidence | My record |
| --- | --- |
| Worked alone / hint / model | |
| First step I repaired and why | |
| New case checked later; date and result | |
| What I can now explain without notes | |
| One remaining question | |

Revisit this idea with your teacher after a delay. Familiarity is different from explaining and using it on a changed case.

---

**WORKBOOK · 12 OF 15**

## Pictorial case practice: Return values and local state

> Return values survive their calls. A function returns a local calculation that the caller stores. Explain the labelled step before checking the changed condition.

**Return value**s survive their calls. A function returns a local calculation that the **caller** stores.

Python 3: def extra(a,b): result=a+b; return result. Then first=extra(2,5); second=extra(4,1); print(first,second).

AI ACADEMY / PYTHON-BRIDGE/9

# Return values and local state

**Start with the supplied facts**

Follow each arrow. Ask which fact allows the next step.

### 1. Trace first call

Its local result is 7; return assigns 7 to first.

### 2. Trace second call

Its new local result is 5; return assigns 5 to second.

### 3. Read caller values

The final print displays 7 5; the second call does not overwrite first.

**CHANGE ONE THING**

Add print(result) after the calls in a fresh script with no global result.

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

Trace the two returned values and explain why first stays seven. What happens if the **caller** tries to print result directly?

**My picture-based explanation and changed-case prediction**

### Record the help used

Support used: alone / hint / worked example. This record is practice **evidence**.

Project connection: For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation.

Use actual Python 3 results to check the **prediction** after reasoning first.

---

WORKBOOK · 13 OF 15

## Project portfolio milestone

> Keep this week's input, deciding step, result and limit together in the Python readiness portfolio record.

Project: Python readiness portfolio

Current milestone: Python readiness portfolio. Keep runnable code, **prediction**s, actual **output**s, one repaired error and an explanation of a function on changed **input**s.

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

| Evidence to keep | My record |
| --- | --- |
| This week's input and deciding step | |

| Evidence to keep | My record |
| --- | --- |
| Prediction and actual result (or labelled paper trace) | |
| One change since the previous checkpoint and why | |
| One error or limit and the next test | |

> **Explain what this week adds to your project. Reuse last week's record rather than starting again.**

---

WORKBOOK · 14 OF 15

## Supported practice and learning evidence

> Try the supported questions before comparing with hints and feedback. Record what you repaired and the evidence you retained.

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

**Return value**s survive their calls. A function returns a local calculation that the **caller** stores.

### Practice 1: explain the deciding evidence

Which labelled step matches this detail? The final print displays 7 5; the second call does not overwrite first.

> 1. Read **caller** values

2. Trace first call

3. Trace second call

**My selected conclusion, deciding fact and explanation**

### Hint · try first

Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

### Feedback · compare after trying

After your attempt, compare with the supported explanation: Read **caller** values The final print displays 7 5; the second call does not overwrite first. Returning passes a value back to the **caller**.

Local names belong to a function call; storing the returned value creates a separate **caller** binding.

**Practice 2: explain the deciding evidence**

### Changed case · predict first

Changed-case practice: Add print(result) after the calls in a fresh script with no global result. Which conclusion is supported by the supplied facts?

1. We can decide the result without examining the changed **input** or the procedure.

2. NameError occurs because that local name is not defined in the **caller**.

3. This one example guarantees correct results for every future **input**.

**My selected conclusion, deciding fact and explanation**

### Hint · try first

Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

### Feedback · compare after trying

After your attempt, compare with the supported explanation: NameError occurs because that local name is not defined in the **caller**. A returned value and the function's internal name are different things.

**Repair a missing building block**

Review Functions and arguments (python-bridge/8). Goal: Distinguish a function definition from a function call. Short practice: Match every argument to a parameter for the two calls.

Why do **output**s differ, and what prints when both calls are removed?. Return to this week and explain what you changed.

Review Loops and trace tables (python-bridge/6). Goal: Trace variable values before and after each loop iteration. Short practice: List the four visited values and their running totals.

Explain why 5 is absent and what prints for range(1,1).. Return to this week and explain what you changed.

Review Expressions and operator order (python-bridge/3). Goal: Evaluate numeric expressions using Python's grouping rules. Short practice: Trace the operation order in both expressions.

Identify which picture step changes with parentheses and predict both results when the multiplier becomes four.. Return to this week and explain what you changed.

### Predict, run and debug

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

### Keep your completed work

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

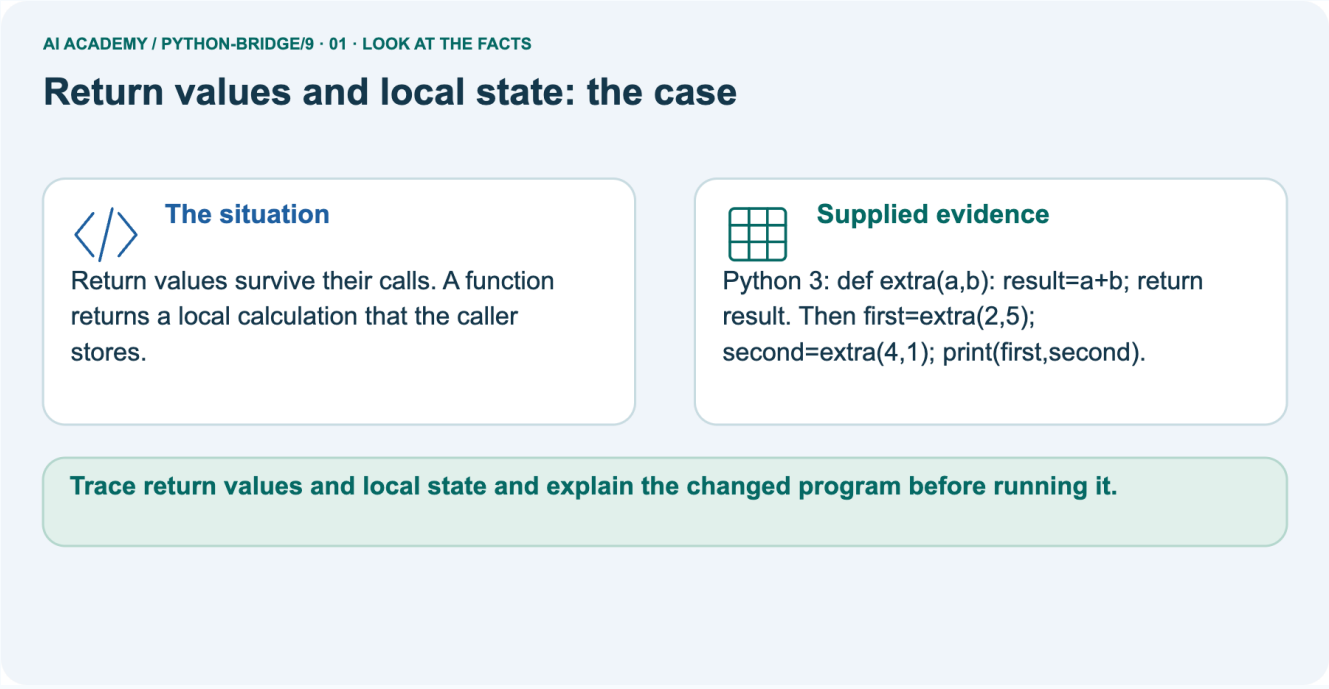
Drafts stay in this browser when storage is available; clear drafts on a shared device.

> **What I repaired, the evidence I kept and the next test**

## Visual concept journey

> Explain each Return values and local state picture from its labels, then record a relevant limit.

### 01. Return values and local state: the case




Follow the labelled picture with the matching explanation. Open the vector diagram

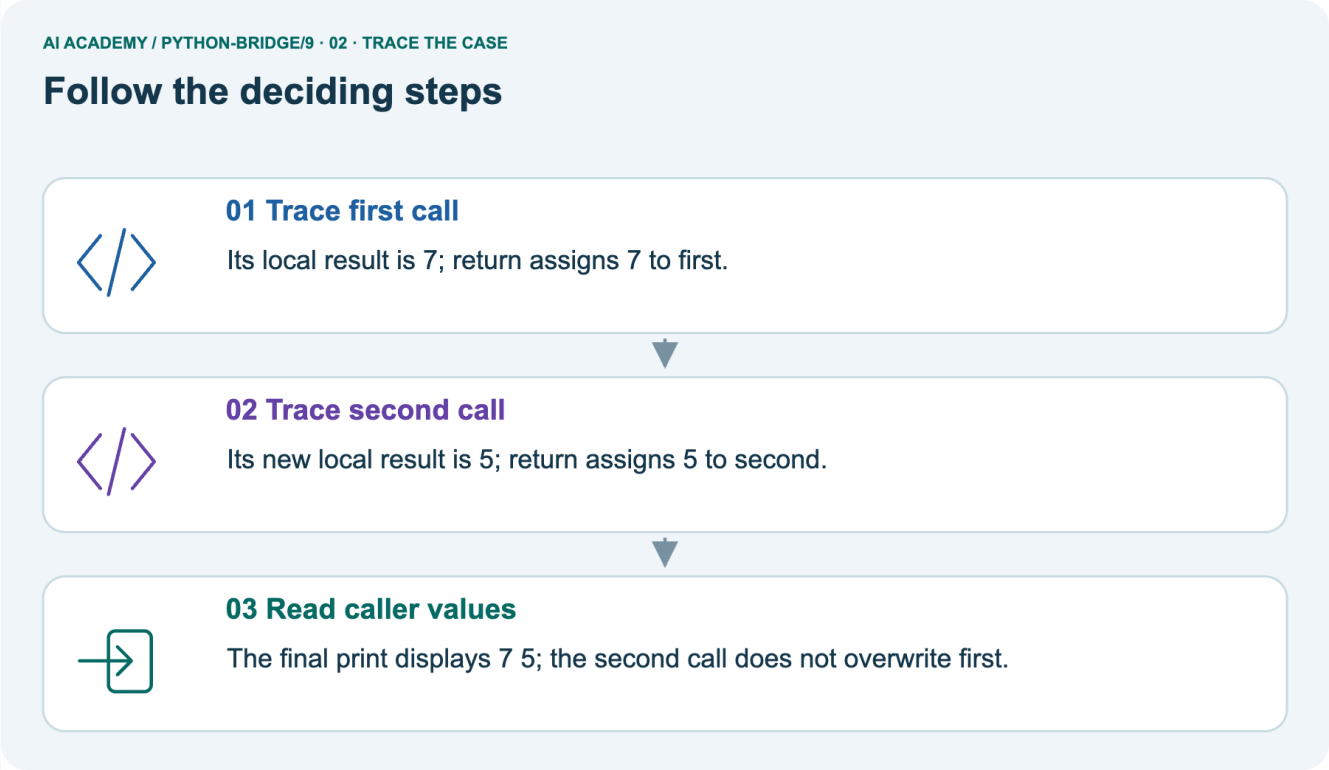
**Return value**s survive their calls. A function returns a local calculation that the **caller** stores. Supplied facts: Python 3: def extra(a,b): result=a+b; return result.

Then first=extra(2,5); second=extra(4,1); print(first,second).

> Point to the given fact that matters most. Explain why it matters.

**My picture explanation, deciding evidence and one limit**

## 02. Follow the deciding steps



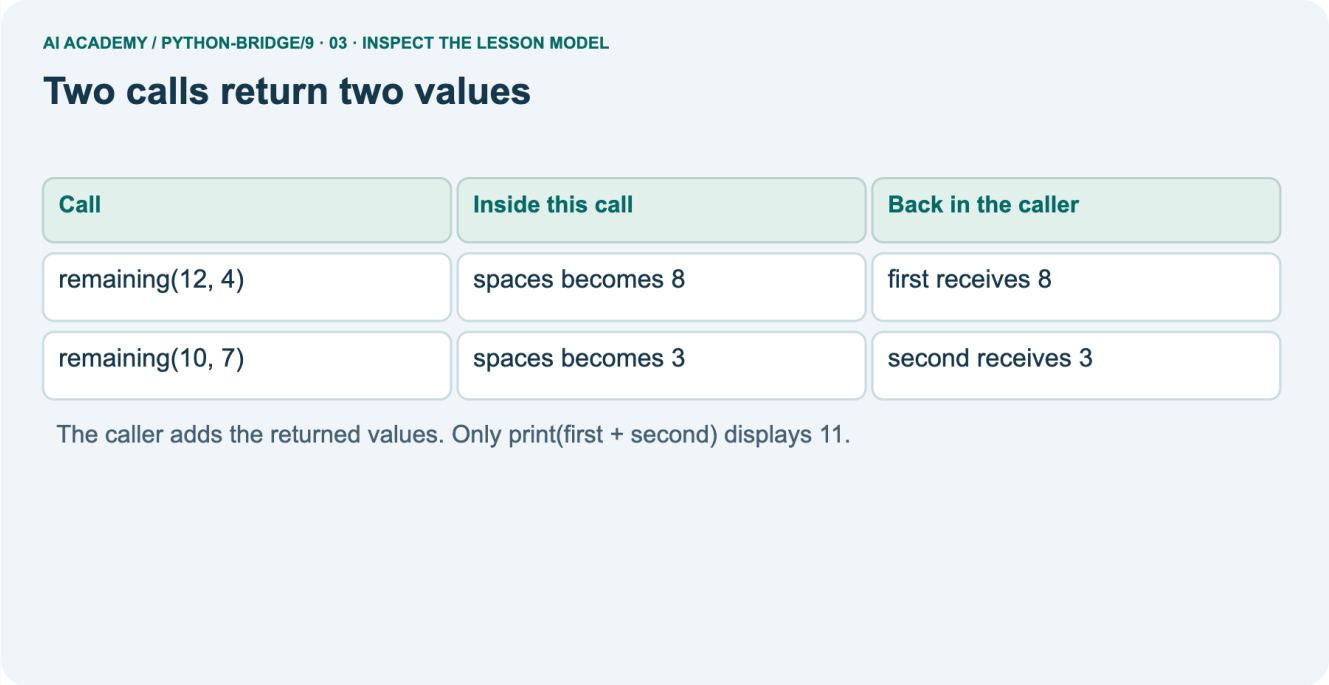

Follow the labelled picture with the matching explanation. Open the vector diagram

Trace first call: Its local result is 7; return assigns 7 to first. → Trace second call: Its new local result is 5; return assigns 5 to second. → Read **caller** values: The final print displays 7 5; the second call does not overwrite first.

Point to each numbered stage and explain which recorded **input** it uses.

**My picture explanation, deciding evidence and one limit**

## 03. Two calls return two values



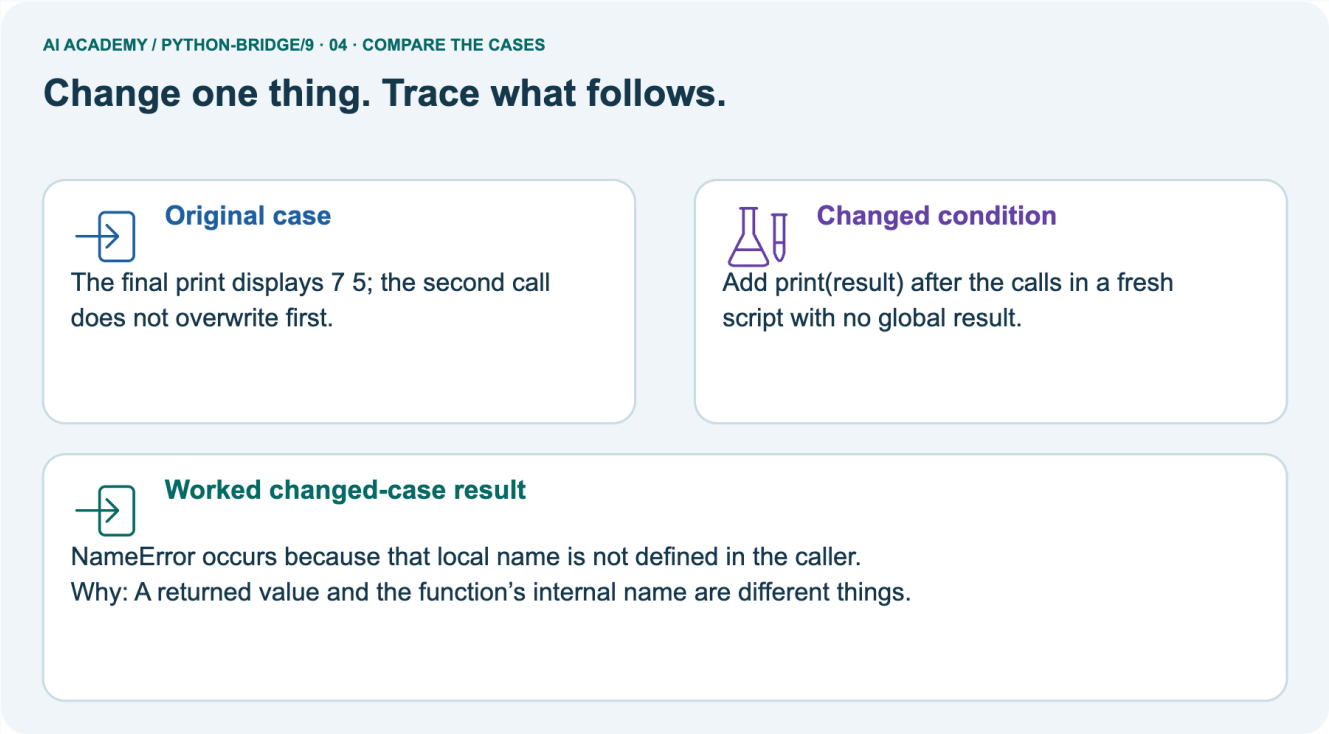

Follow the labelled picture with the matching explanation. Open the vector diagram

Two calls return two values. remaining(12, 4) | spaces becomes 8 | first receives 8. remaining(10, 7) | spaces becomes 3 | second receives 3

Read the labels and values. Explain the relationship the model shows.

**My picture explanation, deciding evidence and one limit**

### 04. Change one thing. Trace what follows.




Follow the labelled picture with the matching explanation. Open the vector diagram

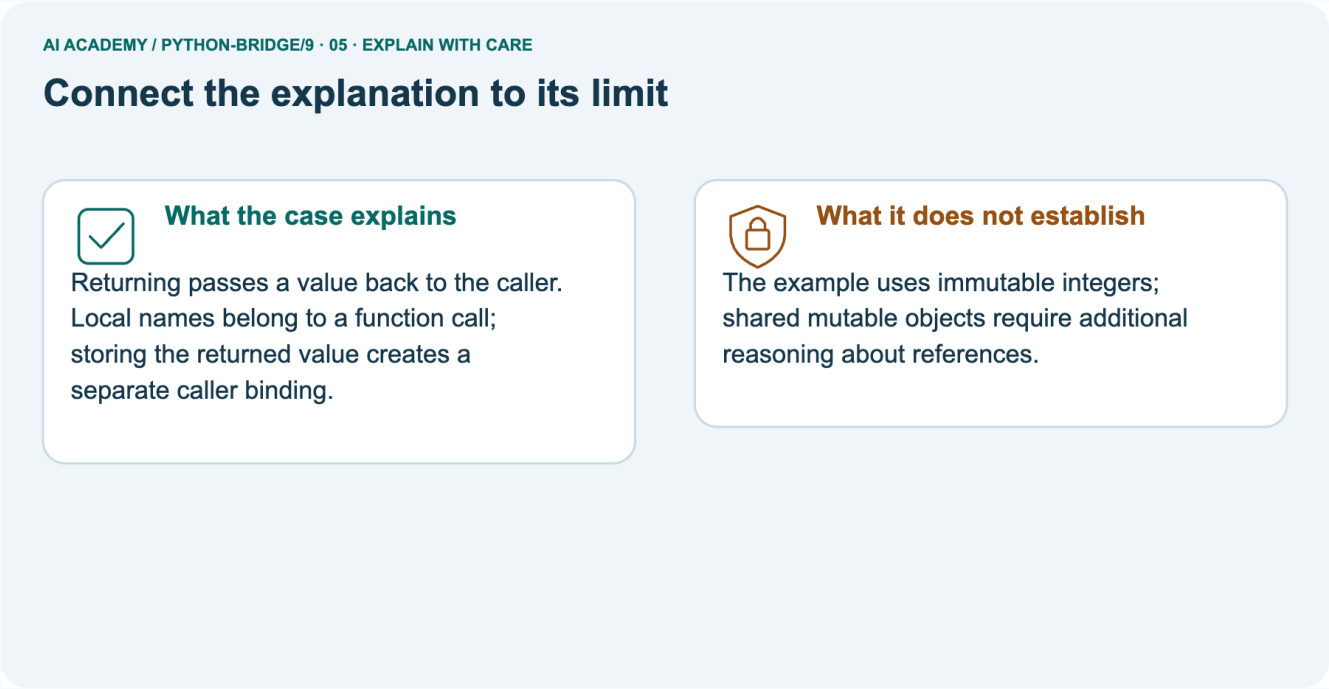
Original case: The final print displays 7 5; the second call does not overwrite first. Changed condition: Add print(result) after the calls in a fresh script with no global result.

Expected result: NameError occurs because that local name is not defined in the **caller**. Reason: A returned value and the function's internal name are different things.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

**My picture explanation, deciding evidence and one limit**

## 05. Connect the explanation to its limit




Follow the labelled picture with the matching explanation. Open the vector diagram

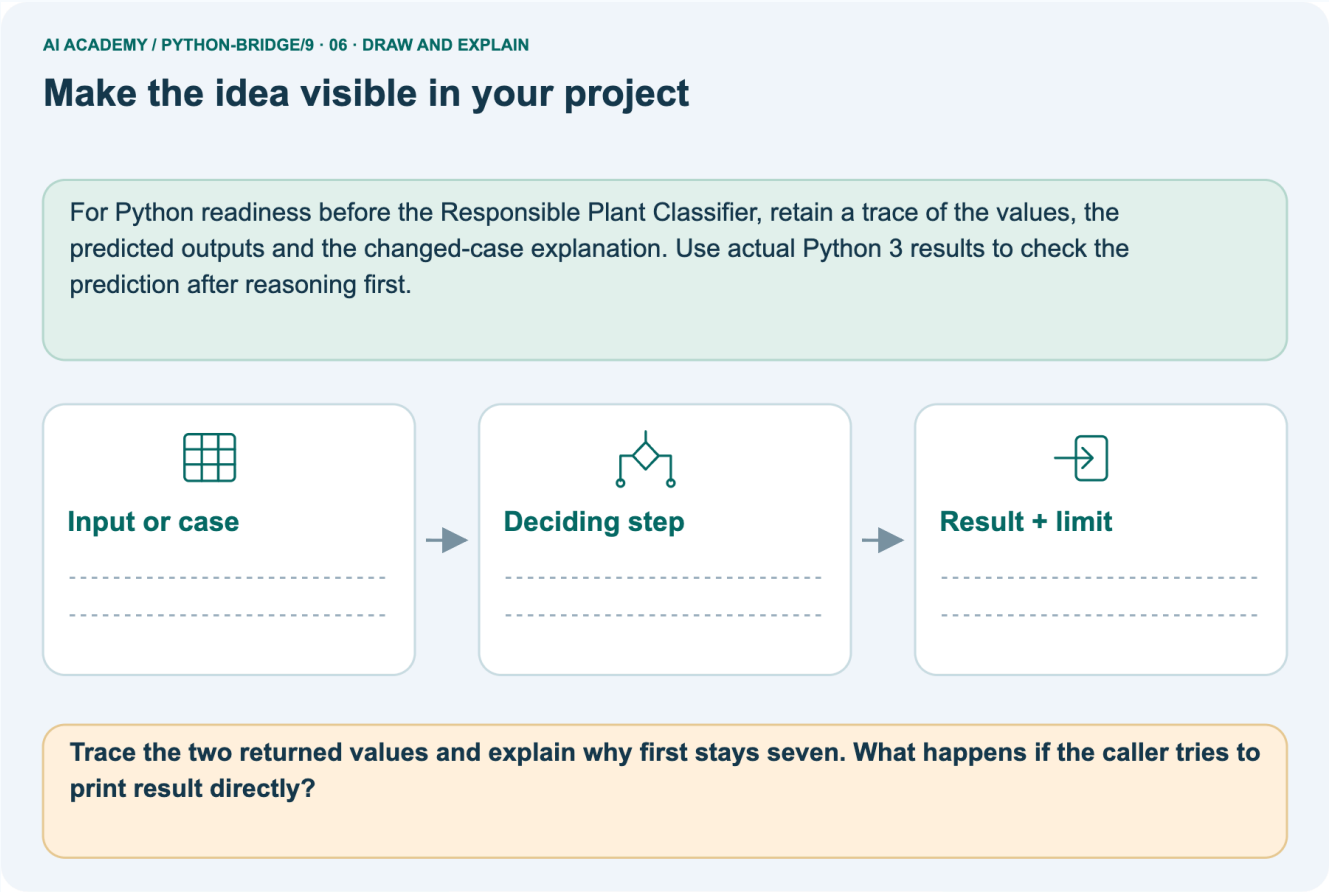
Returning passes a value back to the **caller**. Local names belong to a function call; storing the returned value creates a separate **caller** binding.

Limit: The example uses immutable integers; shared mutable objects require additional reasoning about references.

Explain the case, then point to a limit on the claim.

**My picture explanation, deciding evidence and one limit**

## 06. Make the idea visible in your project




Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Trace the two returned values and explain why first stays seven. What happens if the **caller** tries to print result directly?

Draw your project trace and justify the deciding step using **evidence**.

### My picture explanation, deciding evidence and one limit

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.